Detailed & Simplest Explanation of app.py

A Line-by-Line Guide to the Python Code in Plain, Everyday Language

What is app.py in Simple Words?

Think of app.py as a **smart digital auditor**. It takes any data (binary digits, text messages, or attendance CSV files), divides it using polynomial long division (XOR arithmetic), and produces a short digital fingerprint called a **CRC checksum**. If even a single letter, number, or row changes, the CRC changes completely, allowing the system to instantly catch tampering or corruption!

PART 1: The Tools We Import (Lines 10-14)

```
import streamlit as st
import pandas as pd
import io
import datetime
import os
```

- **import streamlit as st:** Streamlit turns our Python script into an interactive website with buttons, text boxes, and tables without needing HTML or JavaScript. We nickname it st.
- **import pandas as pd:** Used for handling data tables and CSV files (like an Excel sheet in Python). We nickname it pd.
- **import io:** Stands for 'Input/Output'. Helps read and write text files in memory without needing to save temporary files to your hard drive.
- **import datetime:** Provides the computer's current date and time for timestamping our official audit reports.
- **import os:** Stands for 'Operating System'. Helps Python locate files on your computer (like checking if the sample attendance CSV files exist).

PART 2: The CRC Brain Functions (Lines 19-32)

```
def crc(data, gen):
    data = list(data)
    for i in range(len(data) - len(gen) + 1):
        if data[i] == "1":
            for j in range(len(gen)):
                data[i + j] = str(int(data[i + j]) ^ int(gen[j]))
    return "".join(data[-(len(gen) - 1) :])

def text_to_binary(text):
    return "".join(format(ord(char), "08b") for char in text)
```

- **def crc(data, gen):** Creates the CRC function. data is the binary string to divide; gen is the divisor.
- **data = list(data):** Strings cannot be changed in place in Python. Converting to a list lets us flip or modify individual bits.
- **for i in range(len(data) - len(gen) + 1):** Slides the division window from the first bit to the last possible position where the generator fits.
- **if data[i] == '1':** In binary division, we only divide if the leading bit is 1. If it is 0, we slide to the next bit.
- **for j in range(len(gen)): data[i+j] = str(int(...) ^ int(...)):** Performs bitwise XOR (^) bit-by-bit. Identical bits become 0; different bits become 1. No carries or borrows are used!
- **return ''.join(data[-(len(gen) - 1) :]):** Returns only the leftover remainder at the very end of the data. This remainder is the CRC checksum!
- **def text_to_binary(text):** Takes each character, gets its ASCII number using ord(char), turns it into an 8-bit binary string, and joins them together so text can be processed by crc().

PART 3: Page Title & Design (Lines 38-161)

```
st.set_page_config(page_title="CRC Verification...", layout="wide")
st.markdown("<style> ... </style>", unsafe_allow_html=True)
st.markdown("<div class='hero-container'>...</div>", unsafe_allow_html=True)
```

- **st.set_page_config(...):** Sets the browser tab title, puts the shield icon on the tab, and makes the webpage wide-screen.
- **st.markdown('<style>...'):** Adds custom CSS styling (dark slate cards, rounded borders, monospace font for binary numbers, and green/red status badges).
- **st.markdown(' ...'):** Renders the large hero banner at the top of the webpage explaining the system.

PART 4: Sidebar Controls & Step 1: CRC Type Selection (Lines 166-224)

```
crc_type_options = {
    'CRC-3': '1011',
    'CRC-4': '10011',
    'CRC-8': '100000111',
    'CRC-32': '100000100110000010001110110110111'
}
selected_crc_name = st.sidebar.selectbox('Choose CRC Standard:', ...)
gen = generator_input.strip()
crc_len = len(gen) - 1
mode_choice = st.sidebar.radio('Input Mode:', [...])
```

- **crc_type_options:** Stores the standard industry polynomials: CRC-3 (3 bits), CRC-4 (4 bits), CRC-8 (8 bits), and CRC-32 (32 bits).
- **st.sidebar.selectbox(...):** Displays a dropdown menu in the left sidebar for selecting the CRC standard (defaults to CRC-8).
- **st.sidebar.text_input(...):** Fills in the default generator, but lets the user type any custom binary generator they want.
- **crc_len = len(gen) - 1:** Automatically computes the length of the CRC checksum (always 1 less than the generator length).
- **mode_choice = st.sidebar.radio(...):** Lets the user switch between Mode 1 (Binary), Mode 2 (Text), and Mode 3 (Attendance CSV File Audit).

PART 5: Mode 1 - Binary Input (Codeword Method, Lines 228-322)

```
# Sender Side
new_data = sender_bin_data + '0' * crc_len
sender_crc = crc(new_data, gen)
codeword = sender_bin_data + sender_crc

# Receiver Side
remainder = crc(received_codeword, gen)
if int(remainder, 2) == 0:
    extracted_binary = received_codeword[:-crc_len]
    # Status: No Error Detected
else:
    # Status: Error Detected (Data Corrupted)
```

- **Sender Step 1 (Padding):** Appends crc_len zeros to the binary message (e.g. 100100 becomes 100100000).
- **Sender Step 2 (CRC):** Calls crc(new_data, gen) to compute the checksum (e.g. 001).
- **Sender Step 3 (Codeword):** Attaches the CRC to the original data: 100100001. This is the complete transmitted codeword.
- **Receiver Step 1 (Remainder Check):** The receiver divides the received codeword by the generator: remainder = crc(received_codeword, gen).
- **The Golden Rule:** If int(remainder, 2) == 0, the remainder is zero! Status: **No Error Detected**, and the original data is safely extracted.
- **Corrupted Data Check:** If the remainder is non-zero, one or more bits were modified during transmission. Status: **Error Detected**.

PART 6: Mode 2 - Text Input (CRC Comparison, Lines 327-408)

```
# Sender
sender_binary = text_to_binary(sender_text)
sender_crc = crc(sender_binary + '0' * crc_len, gen)

# Receiver
receiver_binary = text_to_binary(receiver_text)
receiver_crc = crc(receiver_binary + '0' * crc_len, gen)

if sender_crc == receiver_crc:
    # Status: No Error Detected (Sender CRC == Receiver CRC)
else:
    # Status: Error Detected (Sender CRC != Receiver CRC)
```

- **Sender Side:** Converts plain text into binary digits using text_to_binary(), adds zeros, and computes sender_crc.
- **Receiver Side:** Converts received text into binary digits and computes receiver_crc.
- **Direct Comparison:** Compares sender_crc == receiver_crc. If identical, the text is verified authentic.
- **Sensitivity:** Even a single character change or capitalization shift completely changes the CRC, immediately catching errors!

PART 7: Mode 3 - Attendance CSV Audit (Lines 412-597)

```
# Original File (Sender)
sender_binary = text_to_binary(sender_file_content)
sender_crc = crc(sender_binary + '0' * crc_len, gen)

# Audited File (Receiver)
receiver_binary = text_to_binary(receiver_file_content)
receiver_crc = crc(receiver_binary + '0' * crc_len, gen)

is_file_intact = (sender_crc == receiver_crc)
# Downloadable Report
st.download_button(label='Download Official Audit Report (CSV)', ...)
```

- **The Classroom Problem:** Solves the instructor dilemma where an attendance CSV file may have rows deleted or attendance values changed.
- **Sender Side (Baseline):** Loads sample_attendance_original.csv, converts it to binary, and calculates the **Reference CRC**.
- **Receiver Side (Audit Target):** Lets the instructor test: (1) Authentic file (PASS), (2) Tampered status where student 103 was changed from Absent to Present (FAIL), or (3) Deleted row where student 105 was removed (FAIL).
- **Audit Verdict:** Compares original vs. audited CRC. A match guarantees 100% authenticity; any mismatch warns of data tampering.
- **Downloadable Report:** Generates a timestamped CSV report certifying the audit result for institutional records.